Practical 10

Aim: Implement value iteration or policy-based decision making in a grid world or similar sequential decision-making environment.

```
from pathlib import Path
import sys

ROOT = Path(__file__).resolve().parents[2]

if str(ROOT) not in sys.path:
    sys.path.insert(0, str(ROOT))

from practicals.practical_10_value_iteration.logic import (
    run_smart_irrigation_value_iteration
)


def display_scenario_results(title, result):
    print("\n" + "=" * 75)
    print(f" {title}")
    print("=" * 75)

    print(f"Crop Type       : {result['crop_type']} ({result['crop_stage']})")
    print(
        f"Soil Moisture  : {result['soil_moisture']}% "
        f"({result['current_moisture_level']})"
    )
    print(f"Discount Factor g : {result['gamma']}")
    print(
        f"MDP Convergence : {result['converged_iteration']} iterations "
        f"(Time: {result['execution_time_ms']} ms)"
    )

    print("\n" + "-" * 75)
    print(" AI IRRIGATION POLICY RECOMMENDATION (Value Iteration)")
    print("-" * 75)

    print(f"Expected State V* : +{result['expected_state_value']}")
    print(
        f"Resource Savings : {result['water_saved_pct']}% water "
```

```
    f"({result['water_saved_liters']}L saved) | "
    f"{result['electricity_saved_kwh']} kWh electricity"
)

print("\n" + "-" * 75)
print(" DECISION EXPLANATION (Why did Value Iteration choose this?)")
print("-" * 75)
print(result['decision_explanation'])

print("\n" + "-" * 75)
print(" ACTION VALUE BREAKDOWN Q(s, a) FOR CURRENT STATE:")
print("-" * 75)

for act in result['action_comparison']:
    win = (
        " [SELECTED OPTIMAL POLICY pi*]"
        if act['action'] == result['recommended_action']
        else ""
    )

    print(
        f" - {act['action']:<25} : "
        f"Q = {act['q_val']:>6.2f} "
        f"(Immediate R = {act['reward']:>5.2f}){win}"
    )

print("\n" + "-" * 75)
print(" FARMING MDP OPTIMAL POLICY MATRIX pi*(s)")
print(" [5 Moisture x 2 Rain States]")
print("-" * 75)

print(
    f" {'Moisture Level':<15} | "
    f"{'No Rain Expected':<22} | "
    f"{'Rain Expected Tomorrow':<22}"
)
print(" " + "-" * 67)

for m_idx, m_name in enumerate(result['moisture_labels']):
    pol_no_rain = result['policy_grid'][m_idx][0]
    pol_rain = result['policy_grid'][m_idx][1]
```

```
        # Display the policy for both rain conditions
        print(
            f" {m_name:<15} | "
            f"{pol_no_rain:<22} | "
            f"{pol_rain:<22}"
        )

    print("\n(* indicates current soil moisture level)")
    print("\n" + "-" * 75)


def main():
    print("=" * 75)
    print(" AI PRACTICAL 10: SMART IRRIGATION SYSTEM (VALUE ITERATION
MDP)")
    print(" Sequential Decision-Making under Uncertainty for Gujarat Farmers")
    print("=" * 75)

    # Check if running interactively with a terminal attached
    is_interactive = sys.stdin.isatty()

    if is_interactive:
        print("\nChoose an option:")
        print(" 1. Enter Custom Field Conditions (User Input)")
        print(" 2. Run Demonstration Scenarios")

        try:
            choice = input("\nEnter choice (1 or 2, default 2): ").strip()
        except EOFError:
            choice = "2"
    else:
        choice = "2"

    if choice == "1":
        print("\n--- Enter Farm Field Conditions ---")
        print("Crops: 1. Cotton | 2. Groundnut | 3. Wheat")

        try:
            c_in = input("Select Crop (1-3, default 1): ").strip()

            crop_map = {
```

```
            "1": "Cotton",
            "2": "Groundnut",
            "3": "Wheat"
        }
        crop = crop_map.get(c_in, "Cotton")

        m_in = input(
            "Soil Moisture % (5 to 95, default 25): "
        ).strip()
        moisture = float(m_in) if m_in else 25.0

        t_in = input(
            "Temperature deg C (15, 30, 35, 45, default 35): "
        ).strip()
        temp = float(t_in) if t_in else 35.0

        print(
            "Water Availability: "
            "1. Low (Scarcity) | 2. Medium (Normal) | "
            "3. High (Abundant)"
        )

        stage = input(
            "Crop Stage (default Flowering): "
        ).strip() or "Flowering"

    except (ValueError, EOFError):
        crop = "Cotton"
        moisture = 25.0
        temp = 35.0
        stage = "Flowering"

    res = run_smart_irrigation_value_iteration(
        crop_type=crop,
        soil_moisture=moisture,
        crop_stage=stage,
        temperature=temp
    )

    display_scenario_results("CUSTOM USER FIELD ANALYSIS", res)
```

```
else:
    # Run the 3 syllabus demonstration scenarios

    print("\n[Executing Syllabus Demonstration Scenarios]...")

    # Scenario 1: Dry Soil + Rain Expected Tomorrow
    res1 = run_smart_irrigation_value_iteration(
        crop_type="Cotton",
        soil_moisture=25.0,
        crop_stage="Flowering"
    )

    display_scenario_results(
        "SCENARIO 1: Dry Soil (25%) + Rain Expected Tomorrow",
        res1
    )

    # Scenario 2: Dry Soil + Scorching Heat (45 deg C) & No Rain
    res2 = run_smart_irrigation_value_iteration(
        crop_type="Cotton",
        soil_moisture=25.0,
        temperature=45.0
    )

    display_scenario_results(
        "SCENARIO 2: Dry Soil (25%) + Scorching Heat "
        "(45 deg C) & No Rain",
        res2
    )

    # Scenario 3: Water Scarcity (Low water availability)
    res3 = run_smart_irrigation_value_iteration(
        crop_type="Groundnut",
        soil_moisture=25.0,
        crop_stage="Vegetative"
    )

    display_scenario_results(
        "SCENARIO 3: Water Scarcity (Low Water Availability)",
        res3
    )
```

```
print("\n" + "=" * 75)
```

```
    print("=" * 75 + "\n")


if __name___== "__main__":
    main()
```

Practical 11

Aim: Represent a real-world problem using propositional logic or first order logic and perform logical inference to derive conclusions using an AI reasoning approach

```
import sys
import time
from pathlib import Path

# Ensure project root is on sys.path
ROOT = Path(__file__).resolve().parents[2]

if str(ROOT) not in sys.path:
    sys.path.insert(0, str(ROOT))


def ascii_safe(text):
    """Sanitize mathematical logic symbols for standard Windows terminals."""
    return (
        str(text)
        .replace("∀", "FORALL ")
        .replace("∧", " AND ")
    )


def print_banner(text):
    print("\n" + "=" * 75)
    print(text)
    print("=" * 75)


def display_knowledge_base():
    print("\n" + "-" * 75)
    print("KNOWLEDGE BASE (KB) RULES [Horn Clauses & First-Order Logic]")
    print("-" * 75)

    for r in FARMING_KNOWLEDGE_RULES:
        if r["id"].startswith("RULE_LEGACY"):
            continue
```

```
        print(f"[{r['id']}] {r['title']}")
        print(f"Propositional : {ascii_safe(r['prop_formula'])}")
        print(f"First-Order  : {ascii_safe(r['fol_formula'])}")
        print(f"Explanation  : {r['explanation']}")
        print()


def run_scenario(title, field_name, crop_type, asserted_facts, query):
    """Execute a logic reasoning scenario and print the deduction trace."""

    print_banner(title)

    # Display the target query
    query_info = PROPOSITIONS_MAP.get(query, {})
    print(
        f"Target Query: [{query_info.get('symbol', query)}] "
        f"{query_info.get('name', query)}"
    )
    print("-" * 75)

    # 1. Forward Chaining
    print(
        "\n[METHOD 1: FORWARD CHAINING INFERENCE "
        "(MODUS PONENS & FOL SUBSTITUTION)]"
    )

    # These functions and variables must be defined in your project.
    fc, steps = forward_chaining(asserted_facts, query=query)

    if not steps:
        print("No rules matched the asserted facts. No new conclusions derived.")
    else:
        for s in steps:
            print(
                f"Step {s['step']} | Triggered: "
                f"{s['rule_id']} ({s['rule_title']})"
            )
            print(
                "Antecedents Met: "
                + ", ".join(s["antecedents_satisfied"])
            )
```

```
        print()


    print(
        f"Forward Chaining Result: Target Query '{query}' is "
        f"{'PROVEN TRUE (+)' if fc['is_proven'] else 'NOT PROVEN (-)'}"
    )
    print(f"Total Facts Derived: {fc['total_derived_facts']}")
    print(f"Execution Time: {fc['execution_time_ms']} ms")

    # 2. SymPy Resolution Refutation Proof
    print(
        "\n[METHOD 2: SYMPY RESOLUTION REFUTATION PROOF "
        "(CNF CONTRADICTION)]"
    )

    res = sympy_crop_resolution_proof(asserted_facts, query=query)
    print(res)

    # 3. Agricultural Advisory Synthesis
    full_run = run_logic_reasoning(
        asserted_facts,
        query=query,
        field_name=field_name,
        crop_type=crop_type
    )

    print("\n[SYNTHESIZED AGRICULTURAL ADVISORY & RECOMMENDATION]")
    print(f"Health Risk Level : {full_run['risk_level']}")
    print(f"Advisory Summary  : {full_run['advisory_summary']}")
    print("-" * 75)


def interactive_mode():
    """Prompt the user for interactive field observations."""

    field_name = "Gujarat_Cotton_FieldA"

    print("\nSelect Crop Type:")
    print("1. Cotton (Kapas)")
    print("2. Groundnut")
    print("3. Wheat")
```

```
crop_choice = input("Enter choice (1-3, default 1): ").strip()

crop_map = {
    "1": "Cotton",
    "2": "Groundnut",
    "3": "Wheat"
}
crop_type = crop_map.get(crop_choice, "Cotton")

symptom_keys = [
    ("LEAF_SPOTS", "Are unusual spots or necrotic lesions visible on leaves?"),
    ("HIGH_HUMIDITY", "Is ambient relative humidity high (>=80%)?"),
    ("LEAF_YELLOWING", "Is leaf yellowing / chlorosis visible?"),
    ("WATERLOGGED", "Is soil around roots waterlogged / standing water?")
]

asserted = []

print("\nAnswer each observable symptom (y/n):")

for key, prompt in symptom_keys:
    ans = input(f"{prompt} [n]: ").strip().lower()

    if ans in ("y", "yes", "true", "1"):
        asserted.append(key)

print("\nSelect Query to Prove:")
print("1. FUNGAL_RISK (Flag Fungal Blight Risk)")
print("2. EXPERT_CONSULT (Escalate to University/KVK Expert)")
print("3. VIRAL_VECTOR_RISK (Cotton Leaf Curl Virus Risk)")

q_choice = input("Enter choice (1-3, default 1): ").strip()

query_map = {
    "1": "FUNGAL_RISK",
    "2": "EXPERT_CONSULT",
    "3": "VIRAL_VECTOR_RISK"
}
query = query_map.get(q_choice, "FUNGAL_RISK")
```

```
    run_scenario(

        f"USER SCENARIO: Field '{field_name}' ({crop_type})",
        field_name,
        crop_type,
        asserted,
        query
    )


def main():
    print_banner(
        "AI PRACTICAL 11: CROP DISEASE ADVISORY & LOGIC REASONING
SYSTEM"
    )
    print("Syllabus: Propositional Logic, First-Order Logic & Logical Inference")
    print(
        "Domain: Gujarat Agriculture "
        "(Cotton, Groundnut, Wheat Disease Advisory)"
    )

    # Display the knowledge base
    display_knowledge_base()

    # Scenario 1: Fungal Disease Risk
    run_scenario(
        "SCENARIO 1: Leaf Spots + High Humidity in Cotton (Fungal Risk)",
        field_name="Field_A",
        crop_type="Cotton",
        asserted_facts=["LEAF_SPOTS", "HIGH_HUMIDITY"],
        query="FUNGAL_RISK"
    )

    # Scenario 2: Viral Vector Risk
    run_scenario(
        "SCENARIO 2: Whiteflies + Curling + Yellowing in Cotton (Leaf Curl Virus)",
        field_name="Field_B",
        crop_type="Cotton",
        asserted_facts=[
            "WHITEFLIES",
            "LEAF_CURLING",
            "LEAF_YELLOWING"
```

```
        ],
        query="VIRAL_VECTOR_RISK"
    )

    # Scenario 3: Waterlogging and Root Rot
    run_scenario(
        "SCENARIO 3: Waterlogged Soil + Chlorosis in Groundnut (Root Rot)",
        field_name="Field_C",
        crop_type="Groundnut",
        asserted_facts=["WATERLOGGED", "LEAF_YELLOWING"],
        query="FUNGAL_RISK"
    )

    # Optional interactive mode
    if sys.stdin.isatty():
        choice = input(
            "\nWould you like to enter your own custom field "
            "observations? (y/n) [n]: "
        ).strip().lower()

        if choice in ("y", "yes"):
            interactive_mode()

    print_banner("PRACTICAL 11 EXECUTION COMPLETED SUCCESSFULLY!")


if __name___== "__main__":
    main()
```

Practical 12

Aim: Implement a simple AI learning model such as k-NN, decision tree, Naive Bayes, or perceptron and compare results on a toy dataset

```
# Initial state: The robot is at location A initial_state
= {"At_A"}

# Goal: The robot must reach location C goal
= "At_C"

# Define all available actions actions
= [
   {
      "name": "Move A to B",
      "pre": {"At_A"},       # Condition: Robot must be at A
      "add": {"At_B"},       # Result: Robot reaches B
      "delete": {"At_A"}     # Remove the old position A
   },
   {
      "name": "Move B to C",
      "pre": {"At_B"},       # Condition: Robot must be at B
      "add": {"At_C"},       # Result: Robot reaches C
      "delete": {"At_B"}     # Remove the old position B
   }
]


# Find an action that can achieve the given goal
def find_action(goal):     for action in actions:
      # Check whether this action produces the goal
if goal in action["add"]:
         return action

   # Return None if no action can achieve the goal
return None


# Create a plan to achieve the goal def
make_plan(goal, state, plan):
   # Step 1: Check if the goal is already achieved
```

```
if goal in state:        return True
```

```
    # Step 2: Find an action that achieves the goal
action = find_action(goal)

    # If no suitable action exists, planning fails
if action is None:          return False

    # Step 3: Achieve all preconditions first
for condition in action["pre"]:
        # Recursively make the required condition true
if not make_plan(condition, state, plan):
            return False

    # Step 4: Execute the action      #
Remove facts that are no longer true
state.difference_update(action["delete"])

    # Add new facts that become true
state.update(action["add"])

    # Store the action in the plan
plan.append(action["name"])

    return True


# Create a copy of the initial state state
= initial_state.copy()

# Create an empty list to store the plan plan
= []

# Run the planning algorithm success
= make_plan(goal, state, plan)

# Display the result if
success:
print("Plan:")
    for step in plan:
print(step)

    print("Final State:", state)
```

```
    print("Goal Achieved!")
else:
   print("Goal cannot be achieved.")
```